

OJ 系统大作业说明

本大作业要求使用 Python 与 FastAPI 构建一个小型但功能完整的 Online Judge（OJ）系统。项目采用分阶段开发方式，主要考查 Python 工程开发、异步编程、REST API、前端交互、进程控制、权限管理、日志记录、数据持久化、自动化测试和 Git 协作规范。

本文档是本次大作业的完整要求。除文档中明确标记为“选做”的内容外，其余内容均为必做要求。验收与评分以本文档为准。

1. 实验目标

完成本项目后，系统应支持以下完整流程：

- 1. 用户通过前端页面完成注册、登录和登出；
- 2. 教师创建并维护编程题目；
- 3. 学生在前端页面查看题目并提交 Python 源代码；
- 4. 系统创建提交记录，并异步启动评测；
- 5. 系统在独立子进程中运行学生代码；
- 6. 系统使用多个测试点进行评测；
- 7. 系统能够识别 AC、WA、RE、TLE 和 SE；
- 8. 系统记录提交状态、最终结果、得分及测试点日志；
- 9. 学生通过前端页面查看自己的提交与经过裁剪的日志；
- 10. 教师通过前端页面完成题目管理或提交管理；
- 11. 教师可以查看全部提交和完整评测日志；
- 12. 管理员可以管理用户、备份和恢复系统数据；
- 13. 服务重启后，题目、用户、提交和日志仍然存在。

2. 技术要求

2.1 开发环境

项目	要求
Python	Python 3.10 或以上
Web 框架	FastAPI
前端实现	Streamlit、原生 HTML/CSS/JavaScript、Vue、React 等均可
ASGI 服务器	Uvicorn
数据模型	Pydantic
自动化测试	pytest
版本管理	Git
基础评测语言	Python

项目	要求
持久化方式	JSON 文件或 SQLite，二选一
接口数据格式	JSON，文件上传接口除外

2.2 异步接口要求

所有 FastAPI 路由处理函数必须使用 `async def` 定义。

评测任务不得长时间阻塞提交接口。创建提交后，接口应先返回提交编号，由后台异步任务继续执行评测。可使用以下任一种方式：

- `asyncio.create_task()`;
- FastAPI `BackgroundTasks`;
- `asyncio.create_subprocess_exec()`;
- `asyncio.to_thread()` 包装阻塞式子进程调用。

本项目不要求实现分布式任务队列，也不要求支持大量并发提交，但提交接口必须能够在评测结束前返回。

2.3 禁止事项

以下实现方式禁止使用：

- 使用 `eval()` 或 `exec()` 执行学生代码；
- 在 FastAPI 主进程中直接执行学生代码；
- 将隐藏测试点或标准答案返回给学生；
- 明文保存用户密码；
- 将虚拟环境、缓存目录、临时评测目录或大体积运行文件提交到 Git；
- 仅在前端隐藏按钮而不在后端执行权限校验；
- 使用静态假数据代替后端接口实现。

2.4 Git 提交规范

Git 提交信息应遵循 Conventional Commits 风格。推荐使用以下前缀：

```
feat: 新增功能
fix: 修复问题
test: 增加或修改测试
refactor: 重构代码
docs: 修改文档
chore: 工程配置或其他维护
```

示例：

```
feat: add problem creation API
feat: implement python judge runner
fix: terminate timed-out subprocess
```

```
test: add hidden testcase permission tests
docs: update deployment instructions
```

不建议使用无实际含义的提交信息，例如：

```
update
修改
final
123
```

3. 项目目录要求

项目至少应包含以下目录和文件。可以增加其他目录，但不得将所有代码集中写在一个文件中。

```
oj_project/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── models/
│   ├── routers/
│   ├── services/
│   ├── repositories/
│   ├── judge/
│   └── utils/
├── data/
├── temp/
├── frontend/
├── tests/
├── report/
│   └── report.md
├── requirements.txt
├── README.md
└── .gitignore
```

各目录建议职责如下：

目录	职责
models/	Pydantic 模型、枚举和数据结构
routers/	FastAPI 路由
services/	业务逻辑
repositories/	JSON 或 SQLite 数据访问
judge/	学生代码运行、输出比较和评测逻辑

目录	职责
utils/	通用工具、时间、编号、脱敏等
tests/	pytest 自动化测试
temp/	评测期间使用的临时目录，不得提交到 Git
frontend/	前端页面、组件、脚本和相关资源；若前端代码放在 app/static/ 等目录，应在 README 中说明
report/	实验报告

项目必须能够通过以下命令启动：

```
pip install -r requirements.txt
uvicorn app.main:app --reload
```

自动化测试必须能够通过以下命令执行：

```
pytest
```

前端的启动方式由所选技术决定，必须在 README 中给出可直接执行的启动命令。

4. 统一接口规范

4.1 API 前缀

除 FastAPI 自动生成的文档接口外，所有业务接口统一使用：

```
/api
```

4.2 统一响应格式

成功响应：

```
{
  "code": 200,
  "message": "ok",
  "data": {}
}
```

失败响应：

```
{
  "code": 404,
  "message": "problem not found",
  "data": null
}
```

要求：

- HTTP 状态码必须与响应体中的 `code` 一致；
- `message` 应能够说明成功或失败原因；
- 没有返回数据时，`data` 使用 `null`；
- Pydantic 参数校验失败时返回 HTTP 422；
- 不得将 Python 完整异常堆栈直接返回给普通用户。

4.3 常用 HTTP 状态码

状态码	使用场景
200	查询、修改、删除成功
201	资源创建成功
202	提交已创建，评测异步进行中
400	请求逻辑不合法
401	未登录
403	已登录但无权限
404	资源不存在
409	唯一字段冲突或状态冲突
422	请求字段校验失败
500	系统内部错误

4.4 时间格式

所有时间字段使用 ISO 8601 格式，建议统一使用 UTC：

```
2026-07-14T10:30:00Z
```

4.5 分页格式

支持分页的接口统一使用：

page: 默认 1, 最小值 1
page_size: 默认 20, 范围 1~100

分页响应:

```
{
  "code": 200,
  "message": "ok",
  "data": {
    "items": [],
    "total": 0,
    "page": 1,
    "page_size": 20
  }
}
```

5. 用户身份与权限约定

5.1 用户角色

系统必须支持以下三种角色:

角色	值	主要权限
学生	student	查看题目、提交代码、查看自己的提交和日志
教师	teacher	管理题目、查看全部提交、查看完整日志、重新评测
管理员	admin	管理用户, 并拥有教师的全部权限

用户还应包含 is_active 字段。is_active=false 的用户不得登录或提交代码。

5.2 认证方式

本项目统一使用基于 Cookie 的 Session 认证。

必须实现:

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET /api/auth/me
```

要求:

- 注册接口创建的用户默认角色为 student;

- 注册接口不得允许客户端自行指定 `teacher` 或 `admin`；
- 登录成功后，服务器在 Session 中保存当前用户编号；
- 登出后清除 Session；
- 密码必须使用安全哈希算法保存，推荐使用 `bcrypt`；
- 不得在接口响应、日志或数据库中返回明文密码；
- 系统首次启动时必须创建一个管理员账号，账号信息写入 README，不得在代码仓库中使用真实个人密码。

5.3 权限判断顺序

处理受保护接口时，应按以下顺序判断：

1. 是否已登录；
2. 用户是否存在；
3. 用户是否处于启用状态；
4. 用户角色是否满足接口要求；
5. 用户是否有权访问目标资源。

前端是否显示按钮不能替代后端权限校验。

基础模块

基础模块共 7 个，均为必做，共 30 分。

Step	名称	主要内容	分值
Step 1	题目管理	题目增删改查、字段校验、隐藏测试点	4
Step 2	Python 自动评测	子进程执行、多测试点、结果判断	4
Step 3	用户与权限	注册登录、角色权限、用户管理	4
Step 4	提交与状态管理	异步提交、状态流转、记录查询	4
Step 5	评测日志	结构化日志、脱敏、权限、审计	4
Step 6	数据持久化	重启保留、备份、恢复	4
Step 7	前端交互	登录、题目浏览、代码提交、结果查询和教师操作	6

Step 1：题目管理

模块目标

实现题目的创建、查询、修改和删除，并保证学生无法获取隐藏测试数据。

前置知识要求

技术点	推荐内容
JSON 或数据库	数据读取、保存和更新
REST API	GET、POST、PUT、DELETE
Pydantic	字段类型、范围和自定义校验
权限控制	学生、教师、管理员
异常处理	404、409、422、500

题目数据结构

每道题目至少包含以下字段：

```
{
  "id": "P1001",
  "title": "A+B Problem",
  "description": "输入两个整数 a 和 b，输出它们的和。",
  "input_description": "一行包含两个整数 a 和 b。",
  "output_description": "输出一个整数，表示 a+b。",
  "samples": [
    {
      "input": "1 2\n",
      "output": "3\n"
    }
  ],
  "constraints": "|a|, |b| <= 10^9",
  "time_limit": 1.0,
  "memory_limit": 128,
  "difficulty": "easy",
  "tags": ["基础", "输入输出"],
  "test_cases": [
    {
      "case_id": "case_01",
      "input": "1 2\n",
      "output": "3\n",
      "score": 50,
      "is_hidden": false
    },
    {
      "case_id": "case_02",
      "input": "-1 2\n",
      "output": "1\n",
      "score": 50,
      "is_hidden": true
    }
  ]
}
```

字段要求：

字段	要求
id	1~32 个字符，仅允许字母、数字、下划线和连字符；全局唯一
title	1~100 个字符
description	非空
input_description	非空
output_description	非空
samples	至少 1 组
time_limit	大于 0，单位为秒
memory_limit	大于 0，单位为 MB；基础模块只要求保存该字段
difficulty	取值为 easy、medium、hard
tags	字符串列表，可为空
test_cases	至少 1 个测试点
case_id	同一道题中唯一
score	非负整数；所有测试点分值之和必须等于 100
is_hidden	是否为隐藏测试点

任务分解

任务 1：题目列表与详情

实现：

```
GET /api/problems
GET /api/problems/{problem_id}
```

权限：

- 学生、教师和管理员均可访问；
- 学生响应中不得包含 test_cases；
- 教师和管理员可以查看完整题目配置。

题目列表至少返回：

```
{
  "id": "P1001",
  "title": "A+B Problem",
  "difficulty": "easy",
  "tags": ["基础", "输入输出"],
  "time_limit": 1.0,
```

```
"memory_limit": 128
}
```

任务 2：创建题目

实现：

```
POST /api/problems
```

权限：仅 **teacher**、**admin**。

成功时返回 HTTP **201**。

必须处理：

- 题目编号重复：**409**；
- 字段缺失或字段范围错误：**422**；
- 测试点为空：**422**；
- 测试点分值总和不等于 100：**422**；
- 保存失败：**500**。

任务 3：修改题目

实现：

```
PUT /api/problems/{problem_id}
```

权限：仅 **teacher**、**admin**。

要求：

- 路径中的 **problem_id** 是被修改题目的编号；
- 不允许通过修改接口将题目编号改为其他编号；
- 题目不存在时返回 **404**；
- 修改后的全部字段仍需通过完整校验。

任务 4：删除题目

实现：

```
DELETE /api/problems/{problem_id}
```

权限：仅 **teacher**、**admin**。

要求：

- 题目不存在时返回 404；
- 若该题已经存在提交记录，仍允许删除题目配置，但历史提交和日志不得被级联删除；
- 历史提交详情应保留原 `problem_id`。

验收要求

至少通过以下场景：

- 创建合法题目；
- 重复创建同一编号；
- 创建缺少必填字段的题目；
- 创建测试点分值不为 100 的题目；
- 查询题目列表；
- 查询不存在的题目；
- 修改题目；
- 删除题目；
- 学生无法看到隐藏测试点；
- 学生无权创建、修改和删除题目；
- 服务重启后题目仍然存在。

Step 2：Python 自动评测

模块目标

接收学生提交的 Python 代码，在独立子进程中运行，并使用题目的多个测试点计算结果和得分。

前置知识要求

技术点	推荐内容
子进程	<code>subprocess</code> 或 <code>asyncio.create_subprocess_exec</code>
异步任务	<code>asyncio.create_task</code> 、 <code>BackgroundTasks</code>
文件操作	临时目录、源码文件、清理
异常处理	超时、返回码、标准错误
字符串处理	换行和行末空格规范化

评测结果

基础模块必须支持以下结果：

状态	全称	含义
AC	Accepted	所有要求的输出均正确
WA	Wrong Answer	程序正常结束，但输出错误
RE	Runtime Error	程序返回非零退出码或发生运行错误

状态	全称	含义
TLE	Time Limit Exceeded	运行时间超过题目限制
SE	System Error	评测器自身发生异常

基础模块不强制实现 MLE。`memory_limit` 字段必须保存和返回，但内存限制可以在进阶安全模块中完成。

输出比较规则

默认评测模式必须执行以下规范化后再比较：

- 1. 将 `\r\n` 和 `\r` 统一转换为 `\n`；
- 2. 删除每一行末尾的空格和制表符；
- 3. 删除文件末尾多余的空行；
- 4. 不忽略行首空格；
- 5. 不忽略行内空格；
- 6. 不允许程序输出题目要求之外的提示语。

例如，标准输出为：

```
3
```

学生输出为：

```
3
```

应判定为 AC。

学生输出为：

```
答案是 3
```

应判定为 WA。

任务分解

任务 1：保存并运行学生代码

评测器应：

- 1. 为每次提交创建独立临时目录；
- 2. 将源代码保存为 `main.py`；
- 3. 使用当前 Python 解释器或配置的 Python 解释器运行；
- 4. 将测试点输入写入程序标准输入；
- 5. 捕获标准输出、标准错误、退出码和运行时间；

- 评测结束后清理临时目录；
- 学生程序输出无法按 UTF-8 解码时，将当前测试点判定为 RE，并记录经过裁剪的错误说明。

禁止使用 `eval()` 和 `exec()`。

任务 2：时间限制

每个测试点均应独立执行，并使用题目的 `time_limit`。

超时时：

- 立即终止对应子进程；
- 当前测试点结果为 TLE；
- 保存测试点日志；
- 可以停止后续测试点，也可以继续评测；但同一项目中必须保持一致；
- 推荐在首次出现 TLE、RE 或 SE 时停止后续测试点。

任务 3：多测试点评测与计分

每个测试点独立产生结果。

测试点得分规则：

测试点为 AC：获得该测试点全部分值
测试点非 AC：该测试点得 0 分

提交总分：

所有 AC 测试点分值之和

最终结果规则：

- 所有测试点均为 AC：最终结果为 AC；
- 出现 SE：最终结果为 SE；
- 否则出现 TLE：最终结果为 TLE；
- 否则出现 RE：最终结果为 RE；
- 否则存在未通过测试点：最终结果为 WA。

任务 4：结构化评测结果

评测服务至少返回以下内部数据，供 Step 4 和 Step 5 使用：

```
{
  "result": "WA",
  "score": 50,
  "total_time": 0.12,
  "cases": [
```

```
{
  "case_id": "case_01",
  "result": "AC",
  "score": 50,
  "time_used": 0.04,
  "exit_code": 0,
  "stdout": "3\n",
  "stderr": ""
},
{
  "case_id": "case_02",
  "result": "WA",
  "score": 0,
  "time_used": 0.08,
  "exit_code": 0,
  "stdout": "0\n",
  "stderr": ""
}
]
```

验收要求

至少准备并正确识别以下代码：

正确代码：

```
a, b = map(int, input().split())
print(a + b)
```

错误答案：

```
print(0)
```

运行错误：

```
print(1 / 0)
```

超时：

```
while True:
    pass
```

还需测试：

- 多测试点；
- 行末空格；
- Windows 与 Linux 换行符；
- 空代码；
- 非 UTF-8 或无法解码的输出；
- 临时目录是否清理；
- 评测器异常时是否返回 SE。

Step 3：用户与权限管理

模块目标

实现用户注册、登录、登出、角色管理和后端权限控制。

前置知识要求

技术点	推荐内容
Session	Starlette SessionMiddleware
密码安全	bcrypt、passlib
权限控制	依赖注入、角色判断
用户模型	唯一用户名、启用状态
审计	操作者、目标和时间

用户数据结构

用户至少包含：

```
{
  "id": "8ec1ef04-9bbb-49a9-b545-8fe03ad59d73",
  "username": "student01",
  "role": "student",
  "is_active": true,
  "created_at": "2026-07-14T10:30:00Z",
  "updated_at": "2026-07-14T10:30:00Z"
}
```

密码哈希可以保存在持久化层中，但不得通过接口返回。

任务分解

任务 1：注册、登录、登出和当前用户

实现：

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET  /api/auth/me
```

注册请求：

```
{
  "username": "student01",
  "password": "example-password"
}
```

登录请求：

```
{
  "username": "student01",
  "password": "example-password"
}
```

要求：

- 用户名 3~32 个字符；
- 用户名全局唯一；
- 密码至少 8 个字符；
- 注册用户默认角色为 `student`；
- 用户名重复返回 `409`；
- 登录失败统一返回 `401`，不得说明是用户名错误还是密码错误；
- 未登录访问 `/api/auth/me` 返回 `401`；
- 被禁用用户登录返回 `403`；
- 登出成功后原 Session 失效。

任务 2：用户列表与详情

实现：

```
GET /api/users
GET /api/users/{user_id}
```

权限：仅 `admin`。

支持分页，用户列表不得返回密码哈希。

任务 3：修改角色和启用状态

实现：

```
PUT /api/users/{user_id}
```

权限：仅 **admin**。

允许修改：

```
{
  "role": "teacher",
  "is_active": true
}
```

要求：

- 角色只能是 **student**、**teacher**、**admin**；
- 用户不存在返回 **404**；
- 不允许管理员通过该接口禁用当前登录的自己；
- 修改结果必须持久化。

任务 4：权限控制

至少保证：

- 学生不能管理题目；
- 学生不能查看其他人的提交；
- 学生不能查看隐藏测试点和完整日志；
- 教师不能修改用户角色；
- 教师可以管理题目和查看所有提交；
- 管理员拥有教师权限和用户管理权限；
- 被禁用用户不能继续执行受保护操作。

验收要求

至少测试：

- 用户注册成功；
- 用户名重复；
- 密码过短；
- 登录成功和失败；
- 登出后无法访问受保护接口；
- 学生调用教师接口返回 **403**；
- 教师调用管理员接口返回 **403**；
- 管理员修改用户角色；
- 禁用用户后无法登录；
- 接口不返回密码或密码哈希。

Step 4：提交任务与状态管理

模块目标

实现代码提交、异步评测、状态流转、结果查询和重新评测。

提交数据结构

每次提交至少包含：

```
{
  "id": "44360932-2952-4362-9252-530c82307016",
  "user_id": "8ec1ef04-9bbb-49a9-b545-8fe03ad59d73",
  "problem_id": "P1001",
  "language": "python",
  "source_code": "a, b = map(int, input().split())\nprint(a + b)",
  "status": "pending",
  "result": null,
  "score": 0,
  "total_time": null,
  "created_at": "2026-07-14T10:30:00Z",
  "started_at": null,
  "finished_at": null
}
```

提交状态

`status` 只能取以下值：

状态	含义
<code>pending</code>	已创建，尚未开始评测
<code>running</code>	正在评测
<code>finished</code>	评测正常结束，包括 AC、WA、RE、TLE
<code>failed</code>	评测系统未能正常完成，通常对应 SE

合法状态流转：

```
pending -> running
pending -> failed
running -> finished
running -> failed
```

不得出现其他状态跳转。

提交结果

`result` 在 `pending` 和 `running` 状态时必须为 `null`。

`finished` 状态的结果可以是：

```
AC
WA
RE
TLE
```

`failed` 状态的结果必须为：

```
SE
```

任务分解

任务 1：创建提交

实现：

```
POST /api/submissions
```

权限：已登录且启用的 `student`、`teacher`、`admin`。

请求：

```
{
  "problem_id": "P1001",
  "language": "python",
  "source_code": "a, b = map(int, input().split())\nprint(a + b)"
}
```

要求：

- `language` 基础模块只允许 `python`；
- `source_code` 不能为空；
- 源代码大小不得超过 64 KiB；
- 题目不存在返回 `404`；
- 成功时创建 `pending` 提交；
- 使用 HTTP `202` 返回提交编号；
- 接口返回后后台继续评测；
- 后台任务开始时改为 `running`；
- 评测结束后改为 `finished` 或 `failed`。

成功响应示例：

```
{
  "code": 202,
  "message": "submission accepted",
  "data": {
    "submission_id": "44360932-2952-4362-9252-530c82307016",
    "status": "pending"
  }
}
```

任务 2：查询单次提交

实现：

```
GET /api/submissions/{submission_id}
```

权限：

- 学生只能查看自己的提交；
- 教师和管理员可以查看任意提交。

学生响应中可以包含自己的 `source_code`，但不得包含测试点输入和标准答案。

任务 3：查询提交列表

实现：

```
GET /api/submissions
```

支持以下可选查询参数：

```
page
page_size
problem_id
user_id
status
result
start_time
end_time
```

权限：

- 学生只能查询自己的提交，服务器应忽略或拒绝学生指定其他 `user_id`；
- 教师和管理员可以查询全部提交。

任务 4：重新评测

实现：

```
POST /api/submissions/{submission_id}/rejudge
```

权限：仅 **teacher**、**admin**。

要求：

- 只允许对 **finished** 或 **failed** 的提交重新评测；
- 其他状态返回 **409**；
- 重新评测时将状态重置为 **pending**；
- 清空当前结果、得分、运行时间、开始时间和结束时间；
- 旧测试点日志可以删除后重建；
- 必须额外写入一条审计记录，说明操作者和重新评测时间。

验收要求

至少测试：

- 创建提交后立即获得 **submission_id**；
- 提交状态按合法顺序变化；
- 评测结果与 Step 2 一致；
- 学生可以查看自己的提交；
- 学生不能查看其他学生提交；
- 教师可以筛选全部提交；
- 不存在的题目无法提交；
- 空代码无法提交；
- 已完成提交不能直接改回 **running**；
- 重新评测权限和状态检查正确。

Step 5：评测日志

模块目标

为每次提交和每个测试点保存结构化评测日志，并根据访问者身份返回不同内容。该模块不仅要求“写日志”，还要求实现日志查询、敏感信息裁剪和访问审计。

日志类型

系统至少包含以下三类记录：

1. 提交级评测摘要；
2. 测试点级评测日志；
3. 审计日志。

测试点日志数据结构

每个测试点日志至少包含：

```
{
  "submission_id": "44360932-2952-4362-9252-530c82307016",
  "case_id": "case_02",
  "result": "WA",
  "score": 0,
  "time_used": 0.08,
  "memory_used": null,
  "exit_code": 0,
  "input_data": "-1 2\n",
  "stdout": "0\n",
  "stderr": "",
  "expected_output": "1\n",
  "message": "output does not match expected answer",
  "is_hidden": true,
  "created_at": "2026-07-14T10:30:01Z"
}
```

要求：

- `memory_used` 未实现时允许为 `null`；
- `input_data` 保存当前测试点输入，仅教师和管理员可通过日志接口查看；
- `input_data`、`stdout`、`stderr` 和 `expected_output` 在持久化前最多保留前 4000 个 Unicode 字符；
- 超过限制时应在结尾追加明确标记，例如 `...[truncated]`；
- 不得在返回给学生的日志中暴露服务器绝对路径、数据库地址、环境变量、完整异常堆栈或其他用户信息。

审计日志数据结构

审计日志至少包含：

```
{
  "id": "4c09ab95-7bf9-4781-9208-d0a9f300e3d2",
  "operator_id": "teacher-user-id",
  "action": "VIEW_FULL_JUDGE_LOG",
  "target_type": "submission",
  "target_id": "44360932-2952-4362-9252-530c82307016",
  "success": true,
  "detail": null,
  "created_at": "2026-07-14T10:40:00Z"
}
```

至少记录以下动作：

```
VIEW_FULL_JUDGE_LOG
REJUDGE_SUBMISSION
UPDATE_USER_ROLE
DISABLE_USER
CREATE_BACKUP
RESTORE_BACKUP
```

日志写入时机

系统至少应在以下时机写入或更新日志：

- 收到提交；
- 评测开始；
- 每个测试点执行结束；
- 发生超时；
- 发生运行错误；
- 发生评测系统错误；
- 整次评测结束；
- 教师或管理员重新评测；
- 教师或管理员查看完整评测日志；
- 管理员执行用户管理、备份或恢复操作。

日志可见性

学生查看自己的提交

学生可以看到：

- 提交编号；
- 最终状态和结果；
- 总得分；
- 总运行时间；
- 每个测试点的 `case_id`、`result`、`score`、`time_used` 和经过裁剪的 `message`；
- 非隐藏测试点的 `stdout` 和 `expected_output`；
- 自己程序产生的、经过路径脱敏和长度截断的 `stderr`。

学生不能看到：

- 隐藏测试点的输入；
- 隐藏测试点的标准输出；
- 隐藏测试点的完整实际输出；
- 服务器绝对路径；
- 完整 Python 堆栈；
- 数据库、环境变量和系统命令；
- 其他用户的日志。

教师和管理员

教师和管理员可以查看：

- 全部测试点；
- 输入数据；
- 标准答案；
- 实际输出；
- 标准错误；
- 退出码；
- 隐藏测试点标记；
- 完整但仍需控制长度的错误信息。

教师或管理员每次查看完整日志时，必须写入审计日志。

路径脱敏要求

返回给学生的错误信息中不得出现真实临时目录，例如：

```
/home/server/oj/temp/44360932/main.py  
C:\oj\temp\44360932\main.py
```

可以替换为：

```
<submission>/main.py
```

或转换为更友好的说明：

```
程序第 3 行发生运行错误。
```

任务分解

任务 1：查询某次提交的日志

实现：

```
GET /api/submissions/{submission_id}/logs
```

权限：

- 学生只能查看自己的提交日志；
- 教师和管理员可以查看任意提交的完整日志；
- 学生访问他人日志返回 403；
- 提交不存在返回 404。

任务 2：教师日志检索

实现：

```
GET /api/logs
```

权限：仅 **teacher**、**admin**。

支持：

```
page
page_size
submission_id
problem_id
user_id
result
start_time
end_time
```

任务 3：审计日志查询

实现：

```
GET /api/audit-logs
```

权限：仅 **admin**。

支持按以下字段筛选：

```
operator_id
action
target_id
start_time
end_time
```

任务 4：日志脱敏与截断

实现统一日志转换函数，避免在多个路由中重复编写字段删除逻辑。

建议区分：

```
to_student_log_view()
to_teacher_log_view()
```

```
sanitize_error_message()
truncate_text()
```

验收要求

至少测试：

- 学生查看自己的日志成功；
- 学生查看他人的日志返回 403；
- 教师查看完整日志成功；
- 学生看不到隐藏输入和标准答案；
- 非隐藏测试点可以展示规定字段；
- 超长输出被截断；
- Linux 和 Windows 绝对路径被脱敏；
- WA、RE、TLE、SE 日志内容正确；
- 教师查看完整日志后产生审计记录；
- 管理员可以筛选审计日志；
- 日志在服务重启后仍然存在。

Step 6：数据持久化、备份与恢复

模块目标

保证服务重启后数据仍然存在，并支持管理员创建备份和从备份恢复。

持久化方式

以下两种方式二选一：

方式 A：JSON 文件

要求：

- 不同类型数据可以保存为独立 JSON 文件；
- 写入时避免因程序中断产生半写入文件；
- 推荐先写临时文件，再原子替换正式文件；
- 文件不存在时初始化为空数据；
- 文件损坏时不得静默清空，应记录错误并返回 500。

方式 B：SQLite

要求：

- 使用 SQLite 文件作为主要数据源；
- 可以直接使用 `sqlite3`，也可以使用 ORM；
- 建议建立用户、题目、测试点、提交、评测日志和审计日志表；
- 外键关系或业务逻辑应保证数据一致性。

不得同时将内存字典作为唯一数据源。服务重启后必须能够恢复全部主要数据。

必须持久化的数据

至少包括：

- 用户；
- 密码哈希；
- 题目；
- 测试点；
- 提交记录；
- 测试点日志；
- 审计日志；
- 备份记录。

备份接口

创建备份

实现：

```
POST /api/admin/backups
```

权限：仅 **admin**。

成功时返回 HTTP **201**：

```
{
  "code": 201,
  "message": "backup created",
  "data": {
    "backup_id": "backup_20260714_103000",
    "created_at": "2026-07-14T10:30:00Z"
  }
}
```

要求：

- JSON 实现应备份全部数据文件；
- SQLite 实现应备份数据库文件；
- 备份中应包含 **manifest.json**，至少记录创建时间、存储类型和文件列表；
- 创建备份时写入审计日志。

查询备份

实现：

```
GET /api/admin/backups
```

权限：仅 **admin**。

恢复备份

实现：

```
POST /api/admin/backups/{backup_id}/restore
```

权限：仅 **admin**。

要求：

1. 备份不存在返回 **404**；
2. 恢复前校验 **manifest.json** 和所需文件；
3. 恢复失败时当前数据不得被破坏；
4. 推荐先将当前数据复制为临时安全副本；
5. 恢复成功后，当前用户 Session 可以失效；
6. 恢复操作必须写入审计日志；
7. 恢复成功后，题目、用户、提交和日志均应与备份一致。

验收要求

必须完成以下完整测试：

1. 创建用户和题目；
2. 完成至少一次代码提交；
3. 确认提交和日志可以查询；
4. 关闭服务；
5. 重新启动服务；
6. 确认全部数据仍然存在；
7. 创建备份；
8. 修改或删除部分数据；
9. 从备份恢复；
10. 确认数据恢复正确；
11. 使用损坏的备份进行恢复；
12. 确认恢复失败且当前数据未被破坏。

Step 7：前端交互

本模块不对页面美观度进行评分，页面布局、配色和视觉样式不作统一要求。页面能够完成规定功能、正确调用后端接口，并清楚反馈操作结果即可。

模块目标

为学生和教师提供可操作的前端界面，使用户能够通过页面完成登录、题目浏览、代码提交、结果查询以及规定的教师管理操作。

技术选型

前端技术不作统一限制，可以选择：

- Streamlit；
- 原生 HTML、CSS、JavaScript；
- Vue；
- React；
- 其他能够正常调用后端 API 并完成规定功能的技术。

前端代码可以单独放在 `frontend/` 目录中，也可以根据所选方案放在 `app/static/`、`app/templates/` 等目录中。项目目录和启动方式必须在 README 中说明。

前后端分离开发时，应正确配置允许的前端来源，并保证登录后的 Cookie Session 能够随请求发送。前端显示的题目、提交、用户状态和评测结果必须来自后端接口。

任务分解

任务 1：登录、当前用户与登出

页面应提供：

- 用户名和密码输入；
- 登录操作；
- 当前登录用户及角色显示；
- 登出操作；
- 未登录状态提示。

登录成功后，后续请求应保持同一登录会话。登录失败、用户被禁用或 Session 失效时，页面应显示明确提示。

任务 2：题目列表与题目详情

学生登录后应能够：

- 查看题目列表；
- 查看题目编号、标题、难度和标签；
- 进入题目详情页；
- 查看题目描述、输入说明、输出说明和公开样例；
- 从题目详情进入代码提交页面。

页面不得展示题目的 `test_cases`、隐藏测试点输入或标准答案。

任务 3：代码输入与提交

题目页面应提供 Python 代码输入区域，并能够调用：

```
POST /api/submissions
```

提交成功后，页面应显示：

- 提交编号；
- 当前提交状态；
- 后续结果查询入口。

代码为空、题目不存在、登录失效或接口调用失败时，应向用户显示对应错误信息。

任务 4：提交状态与评测结果

页面应能够通过自动轮询或手动刷新查询提交状态。

评测完成后，学生至少能够看到：

- 提交编号；
- 题目编号；
- 提交状态；
- 最终评测结果；
- 得分；
- 总运行时间；
- 允许学生查看的测试点信息和错误说明。

页面展示内容必须遵守 Step 5 的日志可见性要求。

任务 5：学生提交记录

学生应能够查看自己的提交列表，并进入某次提交的详情页。

提交列表至少显示：

- 提交时间；
- 题目编号或标题；
- 当前状态；
- 最终结果；
- 得分。

学生不得通过修改请求参数查看其他学生的提交；相应权限仍由后端接口负责校验。

任务 6：教师管理功能

教师登录后，前端至少完整实现以下一类管理功能：

方案 A：题目管理

- 查看题目列表和完整题目详情；
- 创建题目；
- 修改题目；

- 删除题目；
- 显示字段校验和接口错误。

方案 B：提交管理

- 查看和筛选全部提交；
- 查看提交详情和完整评测日志；
- 发起重新评测；
- 显示重新评测后的状态变化。

题目管理和提交管理任选一类完成。

任务 7：页面状态与错误提示

页面至少应处理以下情况：

- 未登录；
- 权限不足；
- 资源不存在；
- 参数校验失败；
- 网络请求失败；
- 后端返回系统错误；
- 提交仍在评测；
- 当前列表为空。

不得使用写死的题目、提交记录或评测结果代替接口返回数据。

验收要求

至少完成以下完整操作流程：

1. 启动后端和前端；
2. 通过前端注册并登录学生账号；
3. 查看题目列表和题目详情；
4. 输入并提交 Python 代码；
5. 查看提交状态变化；
6. 查看最终结果、得分和允许展示的测试点日志；
7. 查看当前学生的历史提交；
8. 使用教师账号登录；
9. 完成题目管理或提交管理中的一类完整操作；
10. 验证未登录、无权限和接口失败时的页面提示；
11. 验证学生页面不会出现隐藏测试点和标准答案；
12. 按 README 中的命令重新启动项目后，前端仍可正常连接后端并完成上述操作。

进阶模块

进阶模块为选做内容，最终最多计 5 分。

评分规则：

- 可以完成一个或多个进阶模块；
- 每个进阶模块单独按 5 分评分；
- 最终进阶得分取所有进阶模块中的最高得分；
- 进阶模块总分上限为 5 分；
- 基础模块未完成的功能不能通过进阶模块重复计分。

Adv 1：Special Judge

模块目标

支持普通比对、严格比对和自定义 Special Judge（SPJ）。

任务分解

任务 1：评测模式

题目增加：

```
judge_mode
```

取值：

值	含义
standard	使用 Step 2 的规范化比较
strict	原始字符串完全一致
spj	调用自定义判题器

任务 2：SPJ 管理

实现教师或管理员上传、替换和删除 SPJ。

SPJ 与题目一一绑定。学生不得查看 SPJ 源码。

任务 3：SPJ 调用约定

SPJ 至少接收：

```
input_data
expected_output
actual_output
```

SPJ 返回：


```
{
  "accepted": true,
  "score": 100,
  "message": "valid solution"
}
```

要求：

- SPJ 本身必须设置超时；
- SPJ 异常或超时应返回 **SE**；
- SPJ 产生的消息也应执行长度限制和脱敏。

Adv 2：安全隔离

模块目标

在基础超时和临时目录措施之上，提高运行学生代码时的隔离能力。

可选实现方式

可以使用：

- Docker；
- 独立低权限 Linux 用户；
- **resource**、**ulimit** 或 cgroups；
- 文件系统与网络限制；
- 其他可说明并验证效果的隔离方案。

要求

至少实现以下四项中的三项：

- CPU 时间限制；
- 内存限制并返回 **MLE**；
- 禁止或限制网络访问；
- 限制文件系统访问范围。

还需：

- 记录沙箱启动失败；
- 评测结束后清理容器或隔离环境；
- 在报告中说明安全边界；
- 提供可重复执行的安全测试。

Adv 3：代码相似度检测

模块目标

对同一道题的多份 Python 提交进行相似度分析，辅助教师发现疑似相似代码。

最低要求

实现：

```
POST /api/problems/{problem_id}/similarity-check
GET  /api/problems/{problem_id}/similarity-reports
```

权限：仅 `teacher`、`admin`。

相似度处理至少包括：

1. 删除空行和注释；
2. 对 Python 代码进行 Token 或 AST 分析；
3. 降低变量名不同对结果的影响；
4. 计算 0~1 之间的相似度；
5. 输出高于阈值的提交对；
6. 保存查重报告。

报告示例：

```
{
  "submission_a": "submission-id-a",
  "submission_b": "submission-id-b",
  "similarity": 0.87,
  "method": "ast",
  "created_at": "2026-07-14T10:30:00Z"
}
```

系统只能给出“疑似相似”结果，不得自动将学生标记为作弊。

代码规范与自动化测试

本部分共 5 分。

代码规范

要求：

- 模块职责清晰；
- 不得将全部逻辑写在 `app/main.py`；
- 核心函数和公开接口具有类型标注；
- 避免复制粘贴重复逻辑；

- 统一处理异常和响应；
- 配置项不得散落硬编码；
- 提供完整 `requirements.txt`；
- 提供可执行的 README；
- `.gitignore` 应排除虚拟环境、缓存、数据库备份和临时评测文件；
- 代码中不得包含真实密码、密钥或个人隐私信息。

自动化测试

必须使用 `pytest`。

至少覆盖：

题目管理

- 创建、查询、修改、删除；
- 重复编号；
- 字段校验；
- 隐藏测试点；
- 权限。

自动评测

- AC；
- WA；
- RE；
- TLE；
- 多测试点；
- 输出规范化；
- 临时文件清理。

用户权限

- 注册和登录；
- 学生、教师、管理员权限；
- 禁用用户；
- 未登录访问。

提交状态

- 创建提交；
- 合法状态流转；
- 非法状态冲突；
- 所有权；
- 重新评测。

日志

- 隐藏字段裁剪；
- 路径脱敏；

- 输出截断；
- 审计记录。

持久化

- 重启后读取；
- 创建备份；
- 恢复成功；
- 损坏备份恢复失败且不破坏现有数据。

实验报告

实验报告共 5 分，提交路径：

```
report/report.md
```

报告至少包含以下内容。

1. 项目概述

说明：

- 项目目标；
- 已完成功能；
- 未完成功能；
- 选择的持久化方式；
- 是否完成进阶模块。

2. 系统架构

说明：

- 路由层；
- 业务层；
- 数据访问层；
- 评测层；
- 日志层；
- 前端层。

建议配系统架构图。

3. 数据设计

说明主要数据模型或数据库表：

- 用户；
- 题目；
- 测试点；

- 提交；
- 评测日志；
- 审计日志；
- 备份记录。

4. 核心实现

重点说明：

- 如何异步启动评测；
- 如何运行和终止学生代码；
- 如何判断 AC、WA、RE、TLE 和 SE；
- 如何比较输出；
- 如何管理提交状态；
- 如何执行权限校验；
- 如何处理隐藏测试点；
- 如何脱敏和截断日志；
- 如何持久化和恢复数据；
- 前端如何维护登录状态、调用后端接口并展示提交结果。

5. API 说明

列出主要接口的：

- 请求方法；
- 路径；
- 权限；
- 请求参数；
- 响应；
- 可能的错误状态码。

6. 测试结果

展示并说明：

- AC、WA、RE、TLE；
- 权限测试；
- 隐藏测试点测试；
- 状态流转测试；
- 日志脱敏测试；
- 重启持久化测试；
- 备份恢复测试；
- 前端登录、题目浏览、代码提交、结果查询和教师操作流程。

7. 问题与解决过程

至少说明两个开发过程中遇到的实际问题，以及解决方式。

8. AI 工具使用说明

如使用生成式 AI 辅助开发，应说明：

- 使用了哪些工具；
- AI 参与了哪些工作；
- 如何验证生成内容；
- 哪些代码或设计由本人修改和确认。

使用 AI 不直接扣分，但提交者必须能够解释自己提交的代码。

提交要求

必须提交

```
app/  
frontend/（或实际使用的前端源码目录）  
tests/  
report/report.md  
requirements.txt  
README.md  
.gitignore
```

前端源码必须完整提交。若前端代码位于 `app/static/`、`app/templates/` 等目录，应在 README 中写明实际位置。

README 必须包含

- Python 版本；
- 安装依赖命令；
- 后端启动命令；
- 测试命令；
- 初始管理员账号的创建或配置方法；
- 使用的持久化方式；
- 数据文件位置；
- 备份文件位置；
- 前端安装与启动命令；
- 已知限制。

不得提交

- `.venv/`、`venv/`；
- `__pycache__/`；
- `.pytest_cache/`；
- 评测临时目录；
- 学生代码临时文件；
- 真实用户数据；
- 真实密码或密钥；

- 无必要的大文件；
- 可通过依赖安装重新生成的第三方包。

总评分

部分	分值
Step 1：题目管理	4
Step 2：Python 自动评测	4
Step 3：用户与权限	4
Step 4：提交与状态管理	4
Step 5：评测日志	4
Step 6：数据持久化、备份与恢复	4
Step 7：前端交互	6
进阶模块	最多 5
代码规范与自动化测试	5
实验报告与线下验收	5
总计	45

重要扣分说明

以下情况将在对应模块中扣分，严重时该模块不得分：

1. FastAPI 路由未使用 `async def`；
2. 使用 `eval()` 或 `exec()` 执行学生代码；
3. 学生能够看到隐藏测试点或标准答案；
4. 学生能够查看或修改他人的提交；
5. 仅在前端实现权限限制；
6. 明文保存密码；
7. 服务重启后主要数据丢失；
8. 无法通过 README 中提供的命令启动项目；
9. 测试依赖手工修改源码或手工准备大量环境；
10. 提交代码与报告内容明显不一致；
11. Git 仓库中包含虚拟环境、临时评测文件或真实隐私数据；
12. 前端无法按 README 启动，或主要页面不能完成规定操作；
13. 无法解释本人提交的核心代码和设计；
14. 网络学堂迟交，每晚交一天（向上取整）扣十分，上限为两天，再迟按照作业未完成处理；
15. 线下验收需提前请假，不接受补假，请假需给出纸质证明（假条、医院诊断等），无故不参加验收按照作业未完成计算。

附录 A：枚举汇总

用户角色

```
student
teacher
admin
```

用户状态

```
is_active = true
is_active = false
```

提交状态

```
pending
running
finished
failed
```

评测结果

```
AC
WA
RE
TLE
SE
```

题目难度

```
easy
medium
hard
```

评测模式

基础模块：


```
standard
```

完成 Adv 1 后:

```
standard  
strict  
spj
```

附录 B：接口汇总

认证

```
POST /api/auth/register  
POST /api/auth/login  
POST /api/auth/logout  
GET  /api/auth/me
```

用户

```
GET /api/users  
GET /api/users/{user_id}  
PUT /api/users/{user_id}
```

题目

```
GET    /api/problems  
GET    /api/problems/{problem_id}  
POST   /api/problems  
PUT    /api/problems/{problem_id}  
DELETE /api/problems/{problem_id}
```

提交

```
POST /api/submissions  
GET  /api/submissions  
GET  /api/submissions/{submission_id}  
POST /api/submissions/{submission_id}/rejudge
```

日志

```
GET /api/submissions/{submission_id}/logs
GET /api/logs
GET /api/audit-logs
```

备份

```
POST /api/admin/backups
GET /api/admin/backups
POST /api/admin/backups/{backup_id}/restore
```

进阶模块接口

```
POST /api/problems/{problem_id}/similarity-check
GET /api/problems/{problem_id}/similarity-reports
```

SPJ 管理接口可自行设计，但必须在 README 和实验报告中完整说明路径、请求格式、权限和状态码。

附录 C：最低验收清单

提交前，应确认以下项目均可实际运行：

- ☐ FastAPI 可以按 README 启动；
- ☐ 所有业务路由使用 `async def`；
- ☐ 用户可以注册、登录和登出；
- ☐ 系统存在学生、教师和管理员角色；
- ☐ 教师可以创建、修改和删除题目；
- ☐ 学生只能查看公开题目信息；
- ☐ 每道题至少包含一个测试点；
- ☐ 测试点总分为 100；
- ☐ 学生可以提交 Python 代码；
- ☐ 提交接口返回 202 和提交编号；
- ☐ 后台评测不会阻塞提交接口；
- ☐ 系统可以识别 AC、WA、RE、TLE、SE；
- ☐ 系统按测试点计算得分；
- ☐ 提交状态遵循规定的状态机；
- ☐ 学生只能查看自己的提交；
- ☐ 教师可以查看全部提交；
- ☐ 系统保存测试点级日志；
- ☐ 学生看不到隐藏测试点和标准答案；

- ☐ 日志中的绝对路径已脱敏；
- ☐ 超长日志已截断；
- ☐ 教师查看完整日志会产生审计记录；
- ☐ 服务重启后数据仍然存在；
- ☐ 前端可以按 README 中的命令启动；
- ☐ 用户可以通过前端登录、查看当前身份并登出；
- ☐ 学生可以通过前端查看题目、提交代码并查询结果；
- ☐ 学生可以通过前端查看自己的历史提交；
- ☐ 教师可以通过前端完成题目管理或提交管理中的一类完整操作；
- ☐ 前端能够显示未登录、无权限、参数错误和请求失败提示；
- ☐ 前端展示的数据均来自后端接口；
- ☐ 学生页面不会展示隐藏测试点或标准答案；
- ☐ 管理员可以创建和恢复备份；
- ☐ 损坏备份不会破坏当前数据；
- ☐ pytest 可以直接运行；
- ☐ README 包含完整安装和运行说明；
- ☐ 实验报告内容与实际实现一致；
- ☐ Git 仓库中不包含虚拟环境、临时文件或敏感信息。